Kubernetes Was Built for Service-Resource Orchestration, MaaS Changes Everything.

于文渊

阿里云百炼平台技术负责人
通义实验室系统工程团队负责人

2025/11/15

Kubernetes's mission

The de-facto platform for service orchestrate over uniform resources

Kubernetes Original Mission

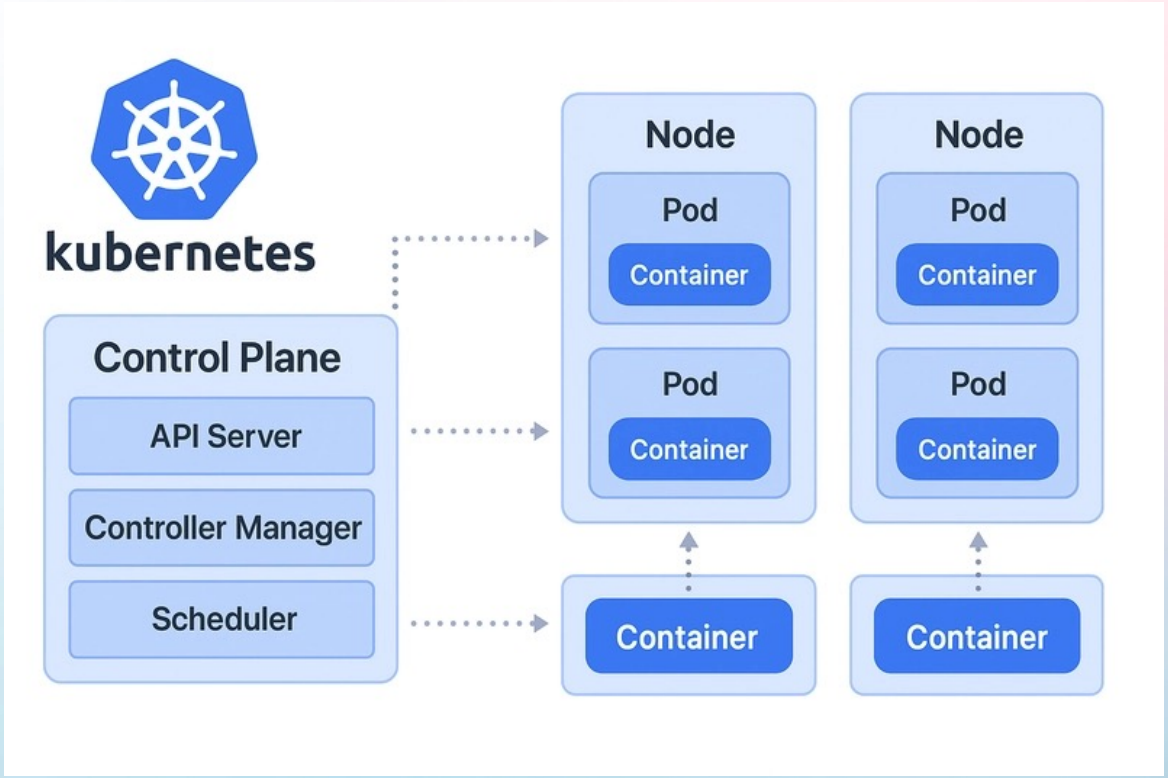
- Control-plane orchestration over uniform compute (Pods, Nodes, Deployments)
- Predictable, long-running web or microservice workloads.

Kubernetes Core Abstractions

- Pod, Deployment, ReplicaSet, Service, Ingress, Autoscaler.
- Scheduler optimizes placement.
- HPA manages scale by metrics.

Targets:

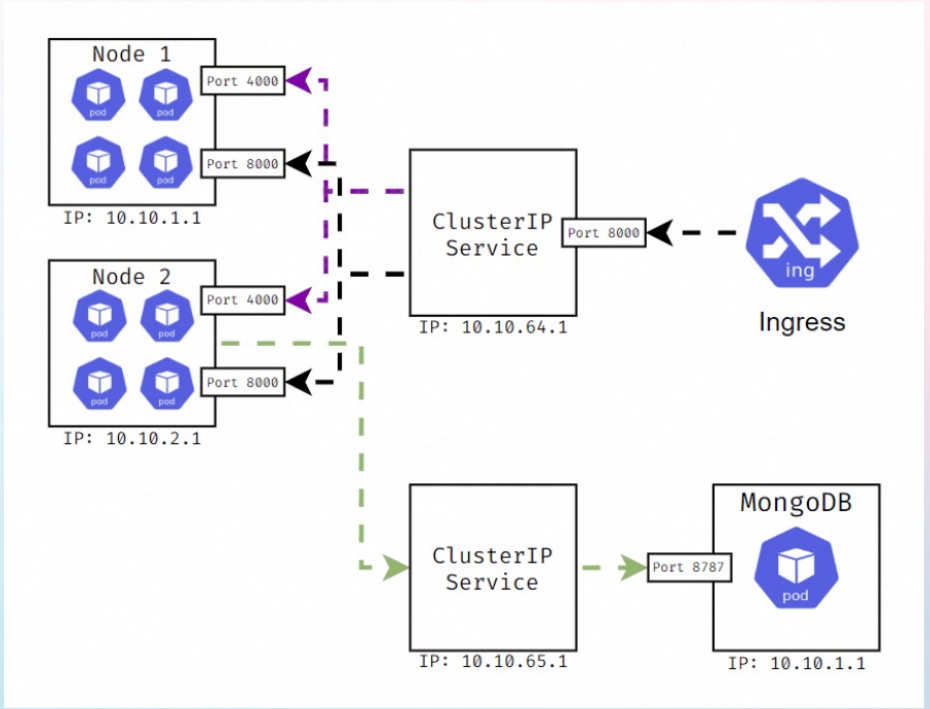
- Design for manage web services / microservices (from Borg)
- Predictable workloads and uniform resource profiles (mainly HTTP/gRPC requests)



Typical web services on Kubernetes

Manage and orchestrate services over uniform resources

- **Ingress:**
  - Accept incoming requests
- **Service:**
  - Route requests to pods
- **Pod Chain & Orchestration:**
  - An application is a service mesh

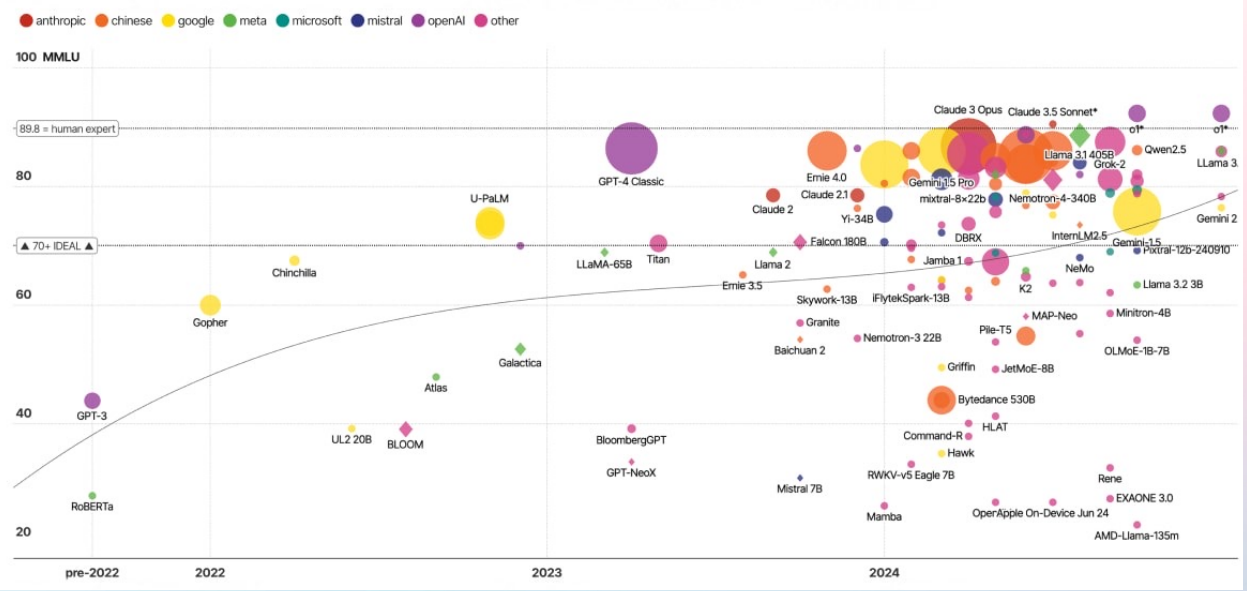


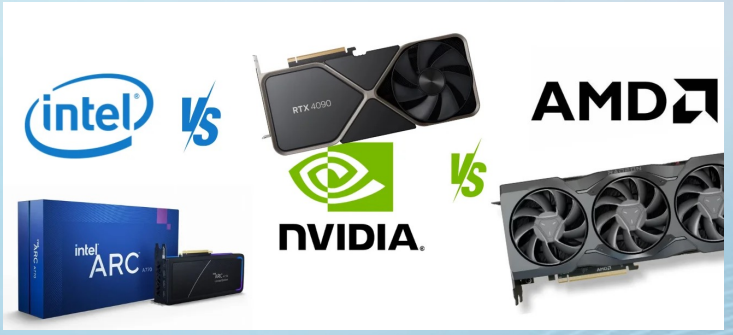
Load-balancing, autoscaling, blue-green rollout... - control-plane-centric.

What makes MaaS different?

The AI infra is more complex than web service infra

- **Various models**
  - LLMs, VLMs, AIGC Models...
  - Thousands of models in different sizes
- **Diverse resources**
  - Different GPUs and accelerators
  - optimized for different model types
- **Complex inference infrastructure:**
  - Cascade pipeline for efficient execution;
  - State (KV-Cache) persistence and session affinity.





K8s Ecosystem Developments

Community attempts for serving GenAI on K8s

WG Serving

- Discuss and enhance the support of inference serving in Kubernetes, including AutoScaling, MultiHost/MultiNode, orchestration and device resource management.

AIBrix

- Provides a Kubernetes CRD for GenAI model serving, encapsulating complexities in ML deployments on Kuberentes.

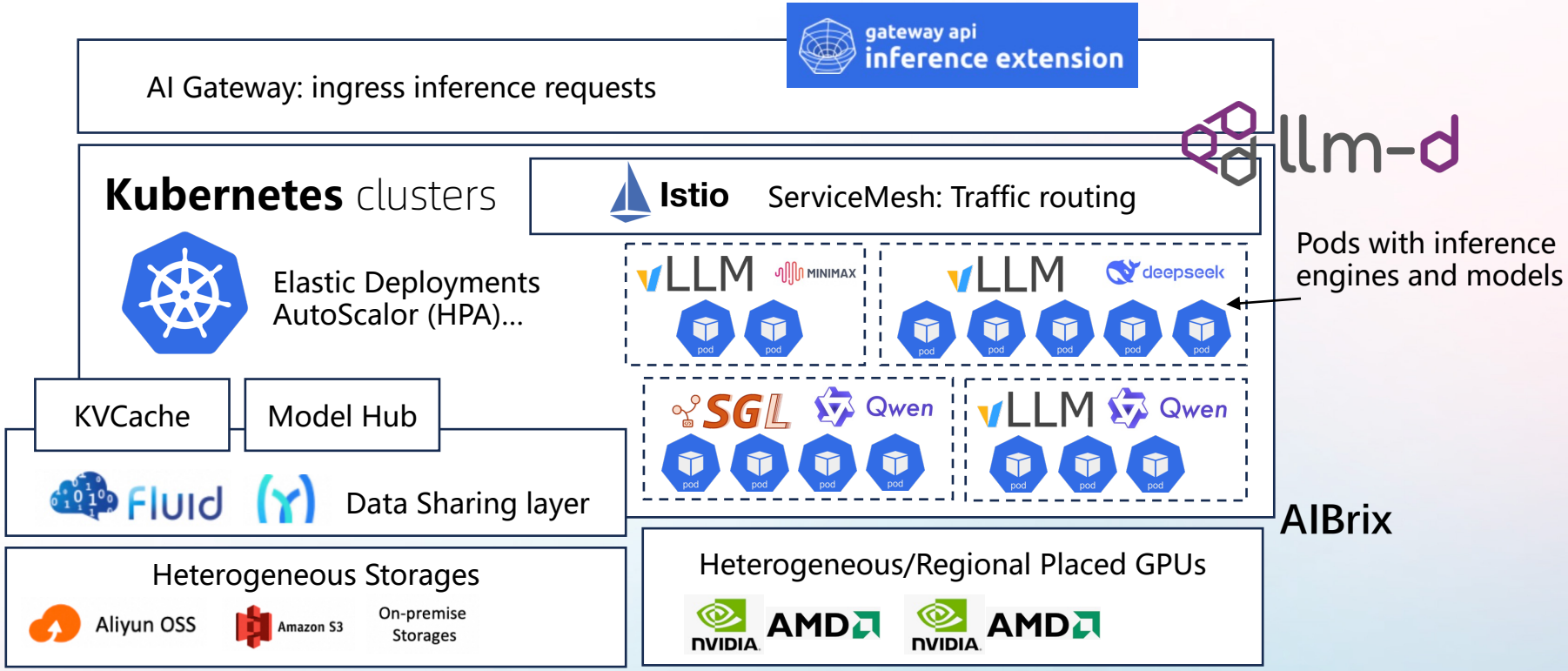
llm-d & Gateway API Inference Extension

- Inference Gateway, request scheduler, routing, and metrics about performance, availability and capabilities.
- Integrating vLLM as the default model inference engine, inference gateway as the request scheduler and balancer, and Kubernetes as the resource orchestrator control plane.

And Many More built on k8s from commnunities (Mooncake, Dynamo, ...)

Reference Architecture “MaaS on K8s”

Request scheduling is not MaaS-native



- **AI Gateway:** handles ingress traffic and routes AI requests into the platform.
- **Model Inference:** A service mesh manages traffic across services, each consisting of pods running inference engines and models.
- **Resource Management:** Kubernetes provides elastic scheduling and scaling of compute resources, including GPUs and CPUs.

Alibaba Cloud Model Studio (DashScope)

阿里云百炼

One of the largest MaaS provider globally

Heterogeneous Resources and Models

- hundreds of thousands of GPUs (Nvidia A800, H20, AMD MI308X, ...) serving at any time
- Geo-distributed clusters (10+ Regions globally)
- Thousands of Models (LLM, VLM, AIGC, Voice, omni……)

Dynamic Request Patterns and Diverse SLOs

- Realtime, Online, Batch, Batch Chat
- Diverse TTFT/TPOT Requirements
- Billions of Requests, Trillions of Tokens each Day
- Millions of AIGC Videos/Images each Day

Routing requests to optimal inference instances for efficiency and SLO compliance.

Challenges for a MaaS Provider

Resource
Cost-Effectiveness

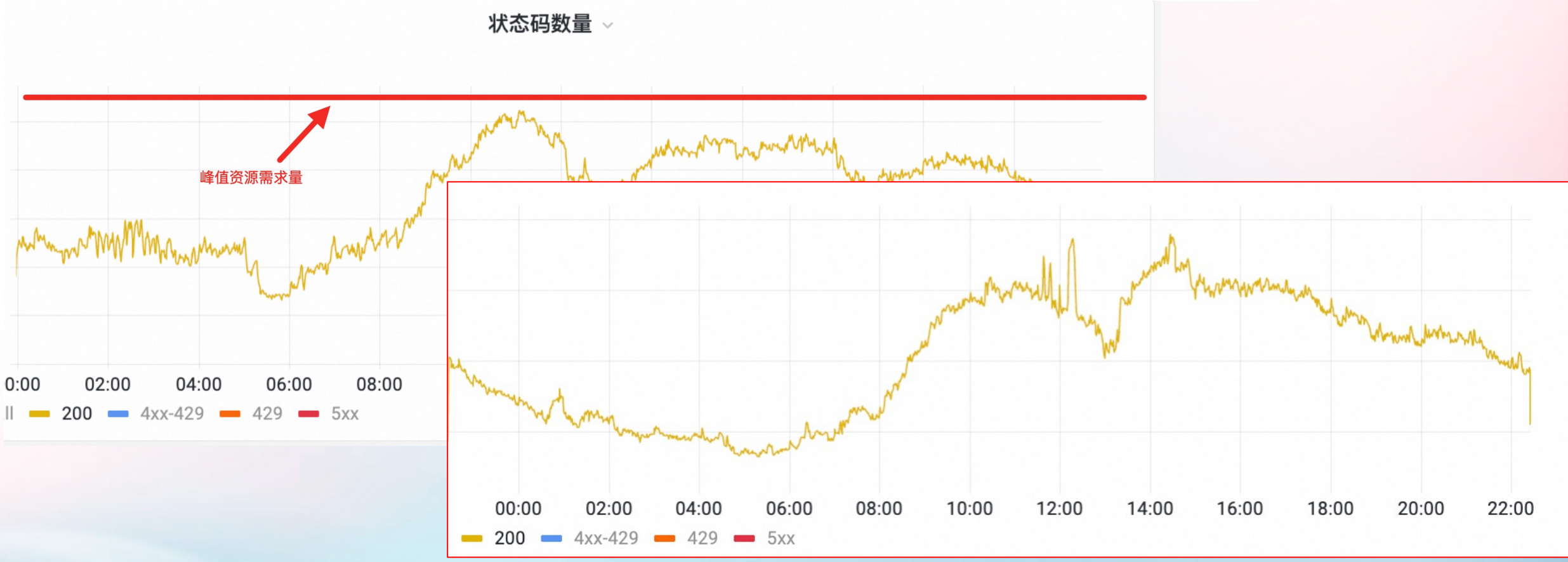
$$\text{每Token成本} = \frac{\text{推理的卡数}}{\text{压测一小时的处理Token数量}} \times \text{GPU的单位小时价格} \div \text{GPU的整体资源利用率}$$

Inference Efficiency

Serving Efficiency

What is Serving Efficiency?

$$\text{GPU的整体资源利用率} = \frac{\text{实际推理的Token数}}{\text{压测单小时单GPU处理Token数量}} \div \text{GPU的总卡时}$$



We model the request distribution as a combination of a Gaussian component (the main market), a Poisson component (the long tail), and occasional black-swan events.

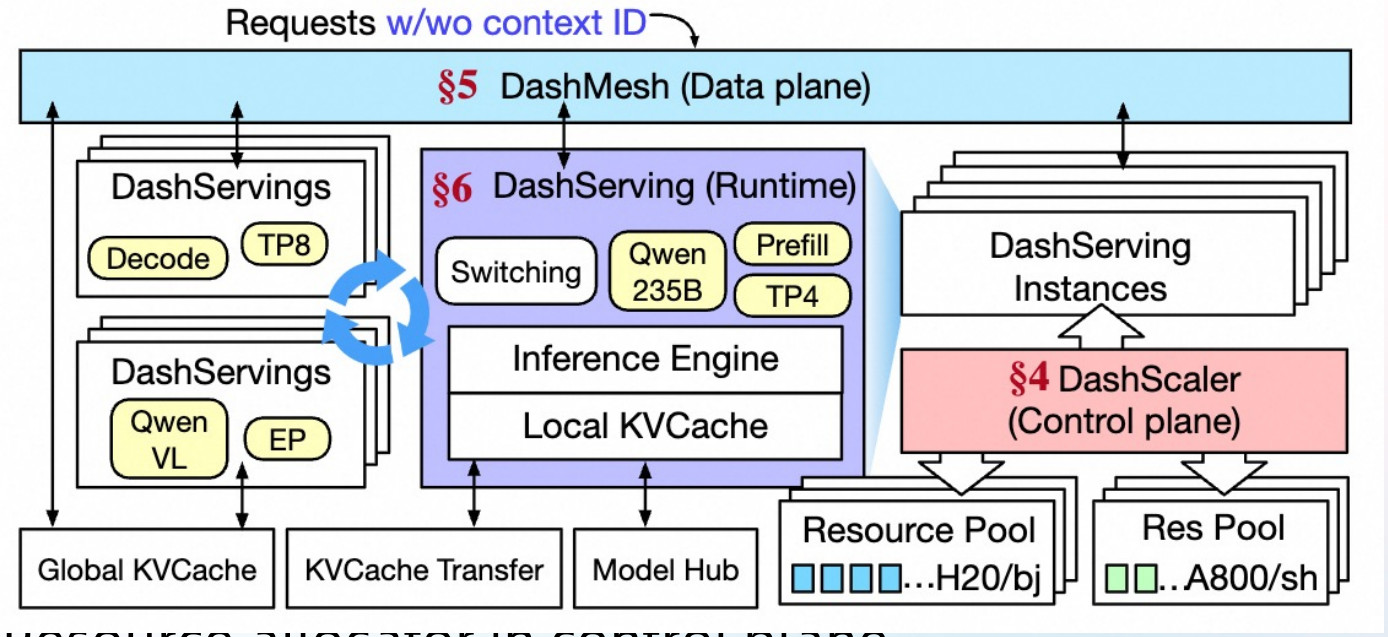
Rethink the boundary of Control vs Data Plane

The line blurs with MaaS:

- **SLO-aware Scheduling**
  - Data Plane? or Control Plane? (Our answer is Data Plane!)
- **KVCache-aware Scheduling**
  - Data Plane? or Control Plane? (Our answer is Data Plane!)
- **PD-Disaggregate Role Scheduling**
  - Not So Sure? (Our answer is Data Plane!)
- **Dynamic Multi-Lora Serving?**
  - Not So Sure? (Our answer is Data Plane, similar as KVCache)
- **AutoScaling?**
  - With K8s it is Control plane, but is it really the best way?
- **Model Switching (qwen-plus -> deepseek), Parallelism (e.g., TP8->EP16) Switching**
  - Control plane?
- **Best Model-Resource fitness?**
  - Control plane?

Why? Compute-Resource allocation is actually happened with each request!

Our Proposed Architecture Overview



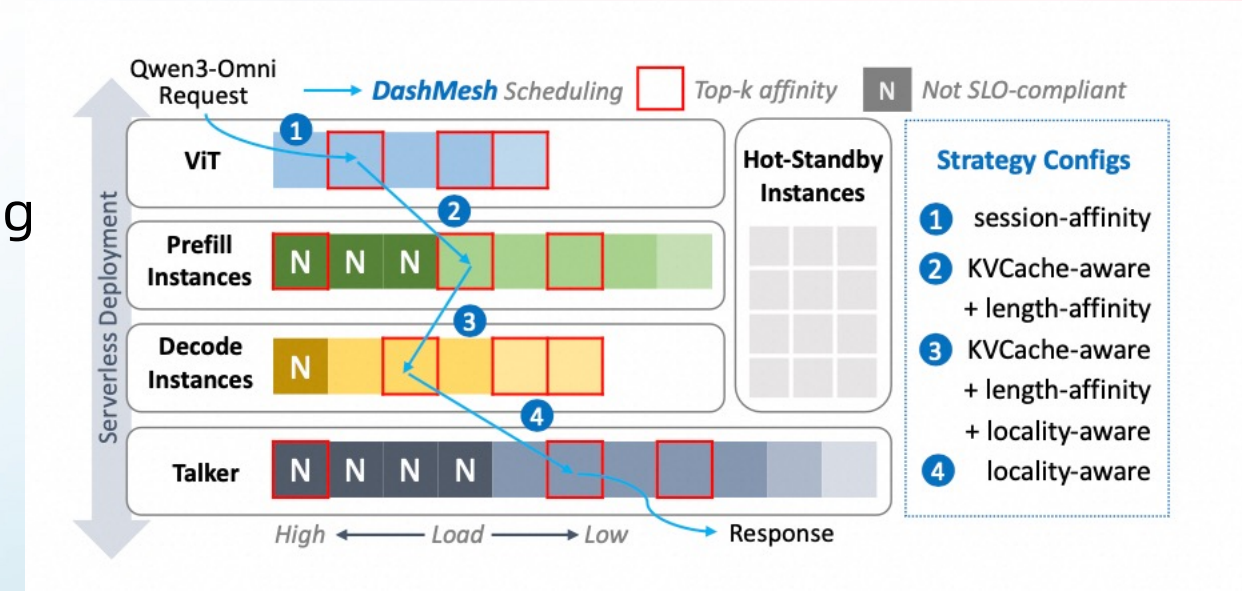
- **DashScaler**: Resource allocator in control plane
  - Throughput-Aware Scheduling
- **DashMesh**: Request scheduler in data plane
  - Request routing: multi-stage, fine-grained scheduling
  - Bin-packing based, prioritizes session-affinity and KVCache reuse
- **DashServing**: Role-agnostic runtime
  - Support model and configuration switching

Design Goals - Rethinking MaaS Architecture

A unified, request-aware MaaS runtime optimized for massive model scale, high QPS, and GPU efficiency.

Model Serving Runtime becomes increasingly capable

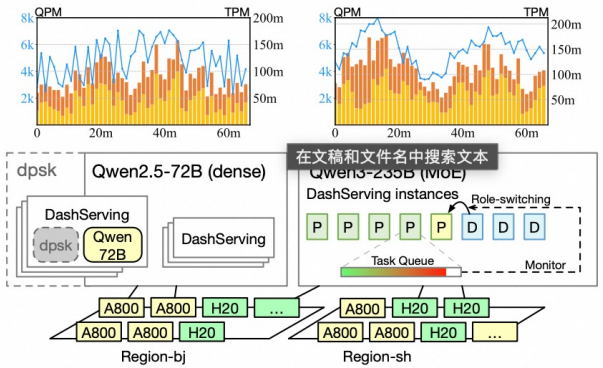
- Fine-grained instance-level async scheduling
- Multi Model Serving / Sleep mode
- KVCache Management / KV Policy / KVStore Connector
- Instance-Level/Request-Level "Role" switching (EPD)

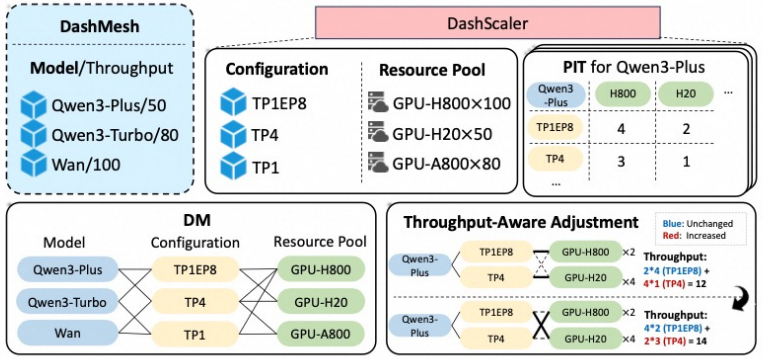


Data Plane

- Move some control-plane logic (routing, resource mapping, scheduling) to Data Plane.
- *BinPacking*
- Define roles (Encoder/Prefill/Decode) and resource-compute bindings dynamically.
- Enable request-level scheduling, KV-aware routing, and SLO-driven execution.

Control Plane






```yaml
# 并行策略配置 (Parallelism Strategy)
tensor_parallelism: 4                    # 张量并行度：模型权重在4个GPU间切分
pipeline_parallelism: 1                  # 流水线并行度：当前为单阶段执行

# 请求处理配置 (Request Handling)
max_batch_size: 32                       # 动态批处理最大请求数
batch_queue_name: "priority_inference_q"  # 批处理队列名称，用于调度系统识别

# 推理引擎规格 (Inference Engine Specification)
engine_spec:
  name: "vLLM"
  params:
    block_size: 16                       # PagedAttention 内存块大小（token）
    gpu_memory_utilization: 0.9          # GPU 显存利用率上限（保留10%缓冲）
    enable_prefix_caching: true          # 启用前缀共享缓存（需与 kvcache_policy 配合）
    max_model_len: 32768                 # 模型最大上下文长度
    quantization: "fp8"                  # 权重量化方案（可选：awq / gptq / fp16 / none）

# 模型资产配置 (Model Assets)
model_weight: "qwen3-coder-plus-xxx"     # 主推理模型标识符
warm_models:                 # 预热模型：启动时加载至显存，降低首请求延迟
  - "qwen3-max-yyy"

# 动态适配器配置 (Dynamic Adapter Management)
active_loras:
```

Control Plane

- Shift to “config” scheduling instead of pod/container-level orchestration.
- Manage model parallelism, expert balance, replicas, LoRA variants, and cache/capacity strategies.
- Containers are too costly and coarse-grained.

What role K8s can paly here...

Open questions remain…

What K8s does on a node is essentially to start / terminate a container.

- Too coarse for MaaS provider, can it be different?

How can we still leverage the K8s ecosystem (Koordinator, AIBrix, llm-d…) while pushing the Serving Efficiency to its physical limit?

KCD GROWING CLOUD NATIVE TOGETHER HANGZHOU

CHINA OpenInfra Days

Q & A

Thanks!